

StonesCamperApp – Projektdokumentation

Stand: Juni 2026

1. Übersicht

StonesCamperApp ist eine progressive Web-App (PWA) für Martin Stein, entwickelt als persönliches Campervan-Tool. Die App läuft vollständig im Browser, benötigt keine Installation und ist auf iPad und iPhone optimiert.

- **Live-URL:** <https://mistermatstone.github.io/StonesCamperApp/>
 - **Code:** GitHub Repository `mistermatstone/StonesCamperApp`
 - **Datenbank:** Firebase Firestore (Projekt: `stonescamperapp`)
 - **Technologie:** Vanilla JavaScript, kein Framework, eine einzige `index.html`
-

2. Technische Architektur

Dateien im Repository

```
/
├─ index.html          ← Die gesamte App (HTML + CSS + JS in einer Datei)
├─ sw.js               ← Service Worker für automatische Updates
├─ favicon.png         ← App-Icon (Browser-Tab + iPad Homescreen)
├─ .nojekyll           ← Verhindert Jekyll-Verarbeitung durch GitHub Pages
├─ docs/               ← Ordner für PDF-Dokumente
│   └─ Anleitung_XY.pdf
└─ ...
```

Firebase Konfiguration

```
apiKey: "AIzaSyDtCpaOGWBSejWQHri_m22qNgQydAF9MP4"
authDomain: "stonescamperapp.firebaseio.com"
projectId: "stonescamperapp"
```

- **SDK:** Firebase Compat v9.23.0 (wird dynamisch nachgeladen)
- **Datenbank:** Firestore, Collection: `camper`, Document: `gewicht`
- **Tarif:** Spark (kostenlos)
- **Sicherheitsregeln:** Unbegrenzt gültig (manuell auf `/camper/{document}` beschränkt)

Datenspeicherung

- **Primär:** Firebase Firestore (Cloud, geräteübergreifend)
- **Fallback:** localStorage (lokal im Browser, greift wenn Firebase offline)
- **Strategie:** App startet sofort mit localStorage-Daten, Firebase synchronisiert im Hintergrund

Service Worker (sw.js)

- Registriert in `index.html` im `<head>`
 - **index.html:** Network First (immer neueste Version laden)
 - **PDFs & Assets:** Cache First (schnelleres Laden)
 - **Firebase/GitHub API:** Kein Cache (immer frisch)
 - **Manuell aktualisieren:** Globale Einstellungen → "App neu laden"
 - **Bei neuem Deploy:** CACHE_NAME Version hochzählen (`v1` → `v2`) falls nötig
-

3. App-Struktur

Screens (HTML-Elemente)

<code>#loading</code>	← Ladescreen beim Start
<code>#home-screen</code>	← Startseite mit Modul-Auswahl
<code>#gewicht-screen</code>	← Gewicht-Modul
<code>#dokumente-screen</code>	← Dokumente-Modul
<code>#pdf-overlay</code>	← PDF-Viewer (Vollbild, über allem)
<code>#confirm-modal</code>	← Bestätigungs-Dialog (Zurücksetzen)

Navigation

- `showScreen(name)` wechselt zwischen Screens

- Mögliche Werte: "home", "gewicht", "dokumente"
 - Jeder Screen hat einen Haus-SVG-Button oben rechts → zurück zur Startseite
-

4. State-Objekt (Datenspeicher)

```
state = {
  // Gespeichert in Firebase + localStorage:
  appName:      "CamperApp",
  appIcon:      null,                // Base64-String des App-Logos
  modules: [
    { id: "gewicht",  name: "Camper Gewicht", icon: "...", iconImage: null, desc
    { id: "dokumente", name: "Dokumente",      icon: "...", iconImage: null, desc
  ],
  categories: [...],
  items: [
    { id, name, weight, category, note, basis: true/false }
  ],
  sets: [
    { id, name, itemIds: [...], note }
  ],
  history: [
    { id, date, totalWeight, selectedWeight, count, itemNames: [...] }
  ],
  docs: [],                // Legacy: manuelle Dokument-Links

  // Nur im Arbeitsspeicher:
  selected:      new Set(),
  currentScreen: "home",
  gewichtView:   "checklist",
  filterCat:     "Alle",
  docsView:      "list",
  syncStatus:    "...",
}
```

5. Gewicht-Modul

Konzept

- **Basisgewichte** = Positionen mit `basis: true` → zählen immer, nicht in Checkliste sichtbar

- **Variable Positionen** = `basis: false` → über Checkliste an/abwählbar
- **Gesamtgewicht** = Basisgewichte + ausgewählte variable Positionen
- **Maximales Gewicht:** 3500 kg (fest im Code: `var MAX_WEIGHT = 3500`)

Berechnungen (gerundet auf 2 Dezimalstellen)

```
baseWeight()      // Summe aller items mit basis:true
selectedWeight()  // Summe aller items mit basis:false UND in state.selected
totalWeight()     // baseWeight() + selectedWeight()
remaining()       // MAX_WEIGHT - totalWeight()
```

Sub-Views (`state.gewichtView`)

Wert	Inhalt
"checklist"	Nur variable Positionen, Sets laden, Fahrt speichern
"items"	Alle Positionen verwalten, Basis-Toggle pro Position
"sets"	Gespeicherte Sets anzeigen, laden, löschen
"history"	Fahrtenverlauf der letzten 20 Fahrten
"settings"	Basisgewicht-Übersicht (read-only), Kategorien, Reset

Basis-Toggle

In der Positions-Verwaltung hat jede Position einen Button:

- **"variabel"** (grau) → `item.basis = false` → erscheint in Checkliste
- **"Basis"** (grün) → `item.basis = true` → immer aktiv, nur in Einstellungen sichtbar

6. Dokumente-Modul

Konzept

PDFs liegen im GitHub Repository unter `/docs/` . Die App liest den Ordner über die GitHub Contents API aus und zeigt alle PDFs automatisch an.

GitHub API

```
var GITHUB_DOCS_API = "https://api.github.com/repos/misttermatstone/StonesCamperAp
```

- Nur `.pdf` Dateien werden angezeigt
- Dateiname als Titel (`.pdf` entfernt, `_` → Leerzeichen)
- Dateigröße wird angezeigt (KB oder MB)

PDF-Viewer (PDF.js)

- Bibliothek: PDF.js v3.11.174 von cdnjs (wird beim ersten PDF-Öffnen geladen)
- **Thumbnail-Leiste:** Alle Seiten als Vorschau, aktive Seite grün markiert
- **Navigation:** Zurück/Weiter Buttons + direkte Seiteneingabe
- **Schließen:** × Button setzt alles zurück inkl. `pdfLoadingTask.destroy()`
- **Wichtig:** `pdfRendering` und `pdfLoadingTask` müssen beim Schließen zurückgesetzt werden

PDF hinzufügen

GitHub → docs/ Ordner → Add file → Upload files → erscheint sofort in der App

7. Globale Einstellungen

Erreichbar über  auf der Startseite:

Einstellung	Beschreibung
App-Name	Anzeigename überall
App-Logo	PNG-Upload als Base64, auch Favicon + iPad-Icon
Module	Name, Beschreibung, Icon-Bild pro Modul
App neu laden	Service Worker deregistrieren + harter Reload
Backup herunterladen	Alle Daten als JSON
Backup wiederherstellen	JSON einlesen

8. Wichtige Funktionen

Kern

```
saveAll()           // localStorage + Firebase (600ms Debounce)
applyData(d)        // Daten in state laden
getPersistedData()  // state → Speicher-Objekt
loadFromLocalStorage() // localStorage → state
saveToLocalStorage() // state → localStorage
syncFromFirebase()  // Firebase → state
loadFirebase()       // Firebase SDK dynamisch laden
showScreen(name)     // Screen wechseln
```

PDF Viewer

```
openPDF(url, title) // PDF öffnen (bricht laufenden Task ab)
closePDF()           // Viewer schließen, alles zurücksetzen
renderPage(pageNum)  // Seite rendern
renderThumbnails()   // Thumbnail-Leiste aufbauen
loadPdfJs(callback)  // PDF.js einmalig laden
```

Export

```
exportJSON()         // JSON herunterladen
importJSON(file)     // JSON einlesen
exportPDF()           // HTML-Export herunterladen
```

Hilfsfunktionen

```
h(tag, attrs, ...children) // DOM-Element erstellen
inlineEdit(value, onSave)   // Inline-Editierfeld
applyAppIcon(dataUrl)       // Favicon + Apple Touch Icon
applyAppTitle(name)         // Browser-Tab-Titel
```

9. CSS-Struktur

Klasse	Verwendung
<code>.hidden</code>	<code>display: none !important</code> — !important wegen ID-Selektoren
<code>.screen</code>	Basis-Container
<code>.header</code>	Dunkelgrüner Header
<code>.home-header</code>	Größerer Header Startseite
<code>.module-btn</code>	Modul-Buttons Startseite
<code>.check-row</code>	Checkbox-Zeile Checkliste
<code>.settings-card</code>	Weißer Einstellungs-Karte
<code>.basis-item</code>	Basisgewicht-Zeile
<code>.basis-total</code>	Grüne Summen-Zeile
<code>.doc-card</code>	PDF-Karte Dokumentenliste
<code>.pdf-overlay</code>	Vollbild PDF-Viewer
<code>.pdf-thumb-bar</code>	Thumbnail-Leiste
<code>.pdf-nav</code>	Navigationsleiste unten
<code>.btn-accent</code>	Grüner Haupt-Button
<code>.btn-ghost</code>	Transparenter Rand-Button
<code>.btn-delete</code>	Roter X-Button
<code>.inline-edit</code>	Klickbarer Text zum Bearbeiten

10. Deployment

Änderungen deployen

1. `index.html` mit Claude bearbeiten
2. Datei herunterladen

3. In GitHub hochladen
4. GitHub Pages deployt automatisch (1-2 Min)
5. In der App: Einstellungen → "App neu laden"

Neue PDFs hinzufügen

GitHub → docs/ → Add file → Upload files

Backup-Routine

Vor größeren Änderungen: Globale Einstellungen → "Backup herunterladen (JSON)"

11. Bekannte Einschränkungen

- **Emojis in JS-Strings** → Safari-Fehler → immer SVG oder Text verwenden
 - **Typografische Anführungszeichen** („") niemals in JS-Strings
 - **window.open()** auf iOS im Standalone-Modus blockiert → PDF.js Viewer als Lösung
 - **Kein Login** → Firebase-Regeln offen für alle (aber nur ein Dokument)
 - **1MB Limit** pro Firestore-Dokument → Base64-Bilder sparsam verwenden
 - **Floating-Point** → alle Berechnungen mit `Math.round(x * 100) / 100`
-

12. Für Claude beim nächsten Mal

Mitschicken:

1. Aktuelle `index.html`
2. Diese Dokumentation
3. Gewünschter Punkt aus der ToDo-Liste

Wichtige Hinweise für Claude:

- Keine Emojis in JS-Strings → SVG-Icons verwenden
- Nach Änderungen Syntaxcheck: `node --check /tmp/test_script.js`
- `.hidden { display: none !important; }` — `!important` ist wichtig!

- Firebase SDK wird dynamisch geladen (nicht im head)
 - App funktioniert auch ohne Firebase (localStorage Fallback)
 - Beim PDF-Viewer: `pdfLoadingTask.destroy()` beim Schließen aufrufen
 - Alle Gewichtsberechnungen mit `Math.round(x * 100) / 100` runden
-

13. ToDo-Liste

DESIGN

- 2x2 Raster für Module statt vertikaler Liste (wie Campinghelfer)
- Hintergrundbild oder dunkleres atmosphärisches Design
- Begrüßungstext mit Datum/Uhrzeit auf Startseite
- Wetter-Widget auf Startseite
- Einheitliche Icon-Sprache (aktuell Mix aus SVG, Emoji, Bildern)
- Farbakzent pro Modul (Gewicht=grün, Dokumente=blau, Tools=orange)
- Ladeanimation zwischen Screens
- Suchfunktion in Positions-Liste

GEWICHT-MODUL

- +/- Buttons bei Positionen (statt nur Checkbox)
- Kategorie-Gruppen als Akkordeon klappbar
- Druckansicht direkt aus der Checkliste
- MAX_WEIGHT in den Einstellungen änderbar machen

DOKUMENTE-MODUL

- Suchfeld über den PDFs
- Kategorien für Dokumente (Fahrzeug, Geräte, Versicherung...)
- Zuletzt geöffnet anzeigen
- PDF-Viewer: Zoom-Steuerung per Button (zusätzlich zu Pinch)

NEUE MODULE (einfach — reine Berechnungen)

- **Gasvorrat-Schätzer** — Flaschengröße + Füllstand + Verbrauchsprofil = Restlaufzeit
- **Batterie & Strom** — Verbraucher togglen, Akkukapazität eingeben, Laufzeit berechnen
- **Reifendruck-Referenz** — Fahrzeugdaten eingeben, Richtwerte anzeigen
- **Urlaubs-Checklisten** — feste Listen für Abreise, Ankunft, Wintercheck

NEUE MODULE (mittel — externe API)

- **KI-Pannenhelfer** — Chat mit Claude API, kennt Fahrzeugdaten
- **Wetter-Widget** — OpenWeatherMap API (kostenlos bis 1000 Aufrufe/Tag)

INFRASTRUKTUR

- Fahrzeugprofil in globalen Einstellungen (Marke, Modell, Leergewicht, max. Zuladung)
- Mehrere Profile (verschiedene Fahrzeuge oder Fahrer)
- Offline-Modus verbessern (vollständige PWA mit Manifest)
- Login mit Google (für sicherere Firebase-Regeln, wenn mehrere Nutzer)